BEST FIT

Completed

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int m, n, i, j;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);
    int block_size[m];
    int block_occupied[m];
    printf("Enter size of each block:\n");
    for (i = 0; i < m; i++) {
        scanf("%d", &block_size[i]);
        block_occupied[i] = 0;
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);
    int process_size[n];
```

Test Cases 2 | Run Sample Cases | Run All Cases

Custom Test Case

> Custom Test

Output Format

- For each process, display whether it is allocated or not.
- If allocated, display the block number, block size, and internal fragmentation.
- If not allocated, display that the process could not be allocated.

Example 1

Sample Input 1

5

100 500 200 300 600

4

212 417 112 426

Sample Output 1

Enter number of memory blocks:

```c
    for (i = 0; i < m; i++) {
        scanf("%d", &block_size[i]);
        block_occupied[i] = 0;
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);
    int process_size[n];
    printf("Enter size of each process:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &process_size[i]);
    }

    for (i = 0; i < n; i++) {
        int best_idx = -1;
```

Test Cases 2 | Run Sample Cases | Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Example 1

Sample Input 1

5

100 500 200 300 600

4

212 417 112 426

Sample Output 1

Enter number of memory blocks:

Enter size of each block:

Enter number of processes:

Enter size of each process:

Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88

Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83

Select Language : C (GCC 9.2.0)

```c
    printf("Enter size of each process:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &process_size[i]);
    }

    for (i = 0; i < n; i++) {
        int best_idx = -1;

        for (j = 0; j < m; j++) {

            if (!block_occupied[j] && block_size[j] >= process_size[i]) {

                if (best_idx == -1 || block_size[j] < block_size[best_idx]) {
                    best_idx = j;
                }
            }
        }
```

Test Cases 2 | Run Sample Cases | Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Example 1

Sample Input 1

5

100 500 200 300 600

4

212 417 112 426

Sample Output 1

Enter number of memory blocks:

Enter size of each block:

Enter number of processes:

Enter size of each process:

Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88

Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83

```c
    printf("Enter size of each process:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &process_size[i]);
    }

    for (i = 0; i < n; i++) {
        int best_idx = -1;

        for (j = 0; j < m; j++) {

            if (!block_occupied[j] && block_size[j] >= process_size[i]) {

                if (best_idx == -1 || block_size[j] < block_size[best_idx]) {
                    best_idx = j;
                }
            }
        }

```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Example 1

Sample Input 1

5

100 500 200 300 600

4

212 417 112 426

Sample Output 1

Enter number of memory blocks:

Enter size of each block:

Enter number of processes:

Enter size of each process:

Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88

Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83

```c
                best_idx = j;
            }
        }
    }

    if (best_idx != -1) {
        block_occupied[best_idx] = 1;
        int fragmentation = block_size[best_idx] - process_size[i];

        printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
               i + 1, process_size[i], best_idx + 1, block_size[best_idx], fragmentation);
    } else {
        printf("Process %d of size %d is not allocated\n", i + 1, process_size[i]);
    }
}

    return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1